

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

# Package Management

---

Unit 6 • Linux (10-151-105)



# Learning Outcomes

---

This unit builds Competency 6 — Manage applications. By the end of the week, you will be able to perform these tasks on both Rocky Linux 10 and Ubuntu Server 24.04:

- **Install packages** add applications and tools to a system using its package manager.
- **Remove packages** uninstall software cleanly and clear out orphaned dependencies.
- **Update packages** apply bug fixes and security patches, for one package or all of them.
- **Evaluate dependencies** understand, inspect, and resolve the software each package relies on.

## WHY BOTH

Package management is where the Red Hat and Debian worlds diverge most — so it is the clearest place to build the dual-ecosystem fluency employers expect.

# Today's Roadmap

---

We'll work through two lessons. The first is about the tools that manage software already known to your system; the second is about getting software onto the system in the first place — safely.

- **Lesson 6.1 • Administer Software with Package Managers** the package families, the manager tiers, and installing, updating, removing, querying, and resolving dependencies.
- **Lesson 6.2 • Acquire and Use Software** where software comes from, downloading it, repositories and their configuration, trust and GPG signing, and sandboxing.

## KEEP IN MIND

Every command today has a Rocky version and an Ubuntu version. Watch for the comparison tables — those side-by-side pairs are the heart of this unit.

# Why Package Management Matters

---

Every system you administer runs software, and software has to be installed, kept current, and eventually removed. Doing that by hand — copying files, tracking versions, chasing down everything a program needs — does not scale and does not stay consistent. Package managers solve this.

- **Consistency** a manager records exactly what it installed, so it can update or remove it cleanly later.
- **Dependency handling** high-level managers pull in everything a program needs automatically.
- **Security** managers verify where software came from and make patching fast and routine.
- **Scale** the same workflow manages one laptop or a thousand servers.

## LESSON 6.1

# Administer Software with Package Managers

---



# The Two Great Families

---

There is no universal Linux package manager. The tools you use are decided by your distribution's lineage. After Linux appeared in 1991, two branches emerged quickly and went on to spawn hundreds of descendants between them.

- **Red Hat family** Red Hat Enterprise Linux, Fedora, CentOS, and Rocky Linux. Uses the .rpm format and the rpm / yum / dnf tools.
- **Debian family** Debian, Ubuntu, Linux Mint, and Kali. Uses the .deb format and the dpkg / apt tools.

## THE POINT

The concept is identical across both — 'install this software and its dependencies.' Learning the second family is mostly a matter of swapping vocabulary.

# Red Hat vs. Debian at a Glance

|                 | Rocky / Red Hat   | Ubuntu / Debian       |
|-----------------|-------------------|-----------------------|
| Package format  | .rpm              | .deb                  |
| Low-level tool  | rpm               | dpkg                  |
| High-level tool | dnf (yum)         | apt (apt-get)         |
| Repo config     | /etc/yum.repos.d/ | /etc/apt/sources.list |
| Admin group     | wheel             | sudo                  |

Same jobs, different dialect. Keep this mapping in mind and most commands translate directly.

# What a Package Actually Is

---

A package is a single bundle containing a program's files plus the metadata a manager needs to install and track it — its name, version, description, and the list of other software it depends on.

- **Files + metadata** everything the program needs, described so a manager can manage it.
- **Dependencies declared** the package states what else it requires to run.
- **Open source & copyleft** most Linux software is freely usable and modifiable, which is why repositories of ready-to-install software exist.
- **Source vs. packaged** you can compile from source for flexibility, but a packaged install is tracked and almost always the better routine choice.



# Four Kinds of Package Managers

| Tier       | What it does                                                                                 | Examples                 |
|------------|----------------------------------------------------------------------------------------------|--------------------------|
| Low-level  | Works directly with one package file. Does NOT resolve dependencies — you fix those by hand. | rpm, dpkg                |
| High-level | Adds repository management and automatic dependency resolution. The everyday tools.          | dnf / yum, apt           |
| Graphical  | A point-and-click front end over the same machinery, for desktop users.                      | GNOME Software, Synaptic |
| Universal  | Distribution-independent formats that bundle dependencies and often sandbox the app.         | Snap, Flatpak, AppImage  |

**FOCUS**

You'll spend nearly all your time in the high-level tier — dnf and apt. The other tiers are worth recognizing, but those two are the daily drivers.

# Low-Level Tools: rpm & dpkg

These operate on a package file already on disk. Precise, but they will not chase down dependencies — when one is missing, they stop and tell you.

| Task               | Rocky — rpm                                      | Ubuntu — dpkg                                    |
|--------------------|--------------------------------------------------|--------------------------------------------------|
| Install a file     | <code>sudo rpm -ivh pkg.rpm</code>               | <code>sudo dpkg -i pkg.deb</code>                |
| Update / reinstall | <code>sudo rpm -Uvh pkg.rpm</code>               | <code>sudo dpkg -i pkg.deb</code>                |
| Remove             | <code>sudo rpm -e name</code>                    | <code>sudo dpkg -r name</code>                   |
| Remove + config    | —                                                | <code>sudo dpkg --purge name</code>              |
| List / info        | <code>rpm -qa</code> / <code>rpm -qi name</code> | <code>dpkg -l</code> / <code>dpkg -s name</code> |

In rpm -ivh: i = install, v = verbose, h = progress hashes.

# High-Level Tools: dnf & apt

The tools you'll live in. Both pull from repositories and resolve dependencies automatically, so one command installs a program and everything it needs.

| Task              | Rocky — dnf                           | Ubuntu — apt                          |
|-------------------|---------------------------------------|---------------------------------------|
| Refresh repo list | (automatic)                           | <code>sudo apt update</code>          |
| Install           | <code>sudo dnf install name</code>    | <code>sudo apt install name</code>    |
| Update one        | <code>sudo dnf update name</code>     | <code>sudo apt upgrade name</code>    |
| Update all        | <code>sudo dnf update</code>          | <code>sudo apt upgrade</code>         |
| Search / info     | <code>dnf list / dnf info name</code> | <code>apt list / apt show name</code> |

The `-y` flag auto-confirms prompts on both — convenient and essential for scripting.

# Installing Software

---

Installation is a single command on both systems — the manager finds the package in a repository, calculates its dependencies, downloads everything, and installs it.

- **Rocky** `sudo dnf install <name>`
- **Ubuntu** `sudo apt update` then `sudo apt install <name>`

## DUAL-DISTRO TRAP

On Ubuntu, apt works from a local cache of the repositories, so run 'apt update' first to refresh that list. On Rocky, dnf checks metadata on its own — 'dnf update' already means 'upgrade my software,' not 'refresh the list.'

# Updating Software

Updating applies bug fixes and security patches. You can update a single package or everything at once — full-system updates are the routine you'll run most.

| Goal               | Rocky — dnf                       | Ubuntu — apt                                             |
|--------------------|-----------------------------------|----------------------------------------------------------|
| Update one package | <code>sudo dnf update name</code> | <code>sudo apt upgrade name</code>                       |
| Update everything  | <code>sudo dnf update</code>      | <code>sudo apt update &amp;&amp; sudo apt upgrade</code> |

SECURITY

Keeping software patched is the single highest-impact routine security control you have. Most compromises exploit a flaw that was already fixed in an available update.

# Holding Back Specific Updates

Sometimes you want to update everything except one package — a newer version breaks your app, or a vendor pins a release. Both ecosystems can exclude or 'hold' a package.

| Goal              | Rocky — dnf                                                     | Ubuntu — apt                           |
|-------------------|-----------------------------------------------------------------|----------------------------------------|
| Skip once         | <code>sudo dnf update --exclude=name</code>                     | (omit from upgrade)                    |
| Hold persistently | add <code>exclude=name</code> to <code>/etc/dnf/dnf.conf</code> | <code>sudo apt-mark hold name</code>   |
| Release the hold  | remove the <code>exclude=</code> line                           | <code>sudo apt-mark unhold name</code> |

Use sparingly — a held package is one you've chosen not to patch, so track why.

# Managing Defaults: update-alternatives

---

On Debian systems, several programs can provide the same capability — different versions of Java, different editors. `update-alternatives` chooses which one is the system-wide default by maintaining symbolic links under `/etc/alternatives/`.

- **See options** `sudo update-alternatives --display java`
- **Choose interactively** `sudo update-alternatives --config java`
- **Add an alternative** `sudo update-alternatives --install /usr/bin/java java <path> <priority>`

## DUAL-DISTRO NOTE

This is a Debian-side tool — there is no direct Red Hat equivalent. Recognize it when you see it on Ubuntu systems.

# Removing Software Cleanly

Removing software frees space and shrinks your attack surface — but it must not break things that depend on it. High-level tools distinguish removing a program from removing its config, and clean up unused dependencies.

| Task             | Rocky — dnf                       | Ubuntu — apt                      |
|------------------|-----------------------------------|-----------------------------------|
| Remove package   | <code>sudo dnf remove name</code> | <code>sudo apt remove name</code> |
| Remove + config  | <code>sudo dnf remove name</code> | <code>sudo apt purge name</code>  |
| Clean up orphans | <code>sudo dnf autoremove</code>  | <code>sudo apt autoremove</code>  |

**CAUTION**

`rpm -e` and `dpkg -r` refuse to remove a package something else needs — and that refusal protects you. Prefer the high-level tools, which understand the whole dependency picture.



# When Dependencies Break

---

A dependency is another package or library a program needs to run. Modern software reuses common components on purpose, so the manager's most valuable job is keeping that web satisfied. Three problems show up most:

- **Missing dependencies** a required package simply isn't installed.
- **Version conflicts** what's installed is the wrong version for what's being installed.
- **Broken dependencies** a package is present but something it needs is missing or corrupted.

## GOOD NEWS

High-level tools (dnf, apt) resolve all of this automatically from repositories — you usually never see a problem. Trouble appears mainly after a low-level install.

# Resolving Dependencies

When a low-level install fails for missing dependencies, step up to the high-level tool to repair it. A few inspection commands help you see what a package needs.

| Goal                             | Rocky / Red Hat                    | Ubuntu / Debian                     |
|----------------------------------|------------------------------------|-------------------------------------|
| List what a package needs        | <code>rpm -qR name</code>          | <code>apt-cache depends name</code> |
| Auto-fix after low-level install | <code>sudo dnf install name</code> | <code>sudo apt install -f</code>    |
| What a binary links against      | <code>ldd /path/to/program</code>  | <code>ldd /path/to/program</code>   |
| Rebuild metadata cache           | <code>sudo dnf makecache</code>    | <code>sudo apt update</code>        |

Ldd works on both families — it shows the shared libraries a compiled program requires.

# Querying What's Installed

Because every manager keeps a database of what it installed, you can ask it questions — essential for auditing, troubleshooting, and answering 'is X installed, and which version?'

| Question               | Rocky — rpm / dnf                                      | Ubuntu — dpkg / apt                                      |
|------------------------|--------------------------------------------------------|----------------------------------------------------------|
| List everything        | <code>rpm -qa</code> / <code>dnf list installed</code> | <code>dpkg -l</code> / <code>apt list --installed</code> |
| Details on one package | <code>rpm -qi name</code> / <code>dnf info name</code> | <code>dpkg -s name</code> / <code>apt show name</code>   |

TIP

These lists get long. Redirect to a file for review: `rpm -qa > softwarelist.txt` — using the redirection you already know.

# Language-Specific Package Managers

System managers handle system software, but most programming languages bring their own manager for code libraries. You'll meet these in development work — they sit alongside dnf and apt, not inside them.

| Manager | Language | Typical commands                                                    |
|---------|----------|---------------------------------------------------------------------|
| pip     | Python   | pip install requests · pip list · pip uninstall — isolate with venv |
| cargo   | Rust     | cargo new · cargo build · cargo install — deps in Cargo.toml        |
| npm     | Node.js  | npm install express · npm update · add -g — deps in package.json    |

**PATTERN, NOT  
COMMANDS**

Each is a manager with its own repository (PyPI, crates.io, the npm registry) and dependency model. Once dnf and apt make sense, these all feel familiar.

# Live Demo

---

## LIVE DEMO

*Install, query, and remove the same package on both pods — side by side.*

- **Rocky** `sudo dnf install tree → tree /etc → dnf info tree → sudo dnf remove tree → sudo dnf autoremove`
- **Ubuntu** `sudo apt update → sudo apt install tree → apt show tree → sudo apt purge tree → sudo apt autoremove`
- **Then break it** download a .deb, `dpkg -i` it to trigger a dependency error, and repair with `sudo apt install -f`

## LESSON 6.2

# Acquire and Use Software

---



# Where Software Comes From

---

Most of the time, software comes from a repository your manager already knows about. But because Linux software is freely distributed, you'll sometimes download a program directly.

- **Distribution repositories** the default and safest source — official, signed, and dependency-aware.
- **Vendor websites** download a package or installer straight from the project (e.g., a .rpm or .deb).
- **Open-source hosts** GitHub and similar sites host source and releases for countless tools.

## DEFAULT TO THE REPO

Prefer your distribution's repositories whenever possible. Direct downloads skip the trust and dependency machinery the repo provides.

# Downloading from the CLI: wget & curl

When you already know a file's URL, fetch it straight from the command line — far easier to script than clicking through a browser.

| Command | What it's for                                                          | Example            |
|---------|------------------------------------------------------------------------|--------------------|
| wget    | Focused downloader; can fetch whole sites recursively. HTTP/S and FTP. | wget -P ~/dl {URL} |
| curl    | Versatile transfer tool (libcurl); many protocols; complex requests.   | curl -o file {URL} |

**RULE OF THUMB**

Reach for wget to grab a file; reach for curl when you need finer control over the request. Both are scripting staples you'll meet again in automation.



# Repositories

---

A repository ('repo') is a storage location full of packages your manager can search, install from, and update against. Repos come in three flavors, distinguished by where they live:

- **Local** stored on the machine itself — simple, but hard to keep consistent across many servers.
- **Centralized internal** hosted on one server on your network that the rest pull from — far easier version control across a fleet.
- **Vendor** maintained on the Internet, usually by the distribution — convenient, but you don't control the versions offered.

# Why Maintain Your Own Repository

---

In an organization, hosting an internal repository is often worth the effort. It gives administrators control that vendor repos can't:

- **Version control** keep beta or unapproved releases away from production.
- **Application control** block unapproved software (games, unvetted tools) from your systems.
- **Vendor control** decide which sources your fleet is allowed to trust.
- **Bandwidth control** many machines update from one local source instead of each hitting the Internet.

# Repository Configuration — Rocky

---

On Red Hat systems, each repository is described by a `.repo` file in `/etc/yum.repos.d/`. A repo entry tells the manager where to fetch packages and whether to verify their signatures.

- **name=** a human-friendly label for the repository.
- **baseurl=** the path to the repo (a `file://` path or an `http://` URL).
- **enabled=1** turns the repository on (0 disables it).
- **gpgcheck=1** verifies package signatures — leave this on.

## AFTER EDITING

Refresh metadata with `sudo dnf makecache` and list active repos with `dnf repolist`.

# Repository Configuration — Ubuntu

---

On Debian systems, repositories are listed in `/etc/apt/sources.list` and in separate files under `/etc/apt/sources.list.d/`. Each entry is a single line.

- **Entry format** `deb {URL} {distro-name} {components}`
- **Add a repo** `sudo add-apt-repository "deb {URL} {distro} main universe"`
- **Disable a repo** comment the line out with a leading `#`

## AFTER EDITING

Always run `sudo apt update` so apt learns about the change before you install or upgrade.

# Trust, GPG Keys & Repository Security

---

Not every repository is trustworthy — some carry beta software, unapproved apps, or even malware. This is why repositories cryptographically sign their packages with a GPG key, and your manager verifies that signature before installing.

- **Authenticity** a valid signature proves the package came from the repo it claims.
- **Integrity** it also proves the package wasn't altered in transit.
- **gpgcheck=1** keeps this verification on — a safety feature, not an obstacle.

## DO THIS

When you hit a 'GPG key error,' import the correct, current key — don't disable checking to make the message go away.

# Troubleshooting Repository Problems

---

When installs or updates start failing, a misconfigured repo is a frequent culprit. Symptoms: packages 'not found,' failed fetches, or errors like 404 Not Found and GPG key error.

- **Check config** list repos (dnf repolist) and inspect the files, or open sources.list and verify URLs.
- **Refresh metadata** sudo dnf makecache / sudo apt update
- **Fix GPG keys** import the missing or updated signing key.
- **Enable / disable** flip enabled= or comment the source line.
- **Clean & rebuild** dnf clean all && dnf makecache / apt clean && apt update

# Sandboxing & Isolation

---

Because software shares the system's resources, one compromised app can endanger the whole machine. Sandboxing runs software in an isolated environment with limited access — a safe space for untrusted, experimental, or under-test software.

- **Virtual machines** a full separate OS; strongest isolation, but resource-heavy.
- **Containers** share the host kernel but isolate the app and its dependencies; lightweight and efficient.
- **Universal formats** Snap, Flatpak, and AppImage bundle an app with its dependencies, with varying sandboxing.

## LOOKING AHEAD

We treat containers and universal formats at awareness level here. Containers get a full hands-on unit later, where you'll build and run them on both distros.

# Security Focus: Trusted Software, Trusted Sources

---



Installing software runs someone else's code with elevated privileges. The package system makes that safe — if you use it deliberately.

- **Keep systems patched** the highest-impact routine control; most attacks exploit already-fixed flaws.
- **Trust the source** prefer official repos; every third-party repo is a new party you trust with root.
- **Verify signatures** let GPG checking confirm authenticity and integrity — don't switch it off.
- **Install only what you need** less software is less to patch and defend; autoremove the orphans.
- **Isolate the risky** sandbox untrusted software to limit the blast radius if it misbehaves.

*Why it matters: the supply chain for your software is an attack surface — treat where it comes from with the same care as what it does.*



# Live Demo

---

## LIVE DEMO

*Inspect repositories and verify where a package really comes from.*

- **Rocky** `dnf repolist` → `cat` a file in `/etc/yum.repos.d/` → note `enabled=` and `gpgcheck=` → `dnf info <pkg>` to see its repo
- **Ubuntu** `cat /etc/apt/sources.list` → `ls /etc/apt/sources.list.d/` → `sudo apt update` → `apt show <pkg>` to see its origin
- **Trust** point out the GPG settings and discuss what a signature failure would mean before an install

# This Week's Lab

---

In your NETLAB+ pod you'll put all of this into practice on both Rocky Linux 10 and Ubuntu Server 24.04, starting from the week's known-good snapshot.

- **Install, update & query** use dnf and apt to add software, patch it, and inspect what's installed.
- **Work low-level** use rpm and dpkg directly, and see where they stop.
- **Resolve a dependency** trigger and repair a broken dependency with the high-level tools.
- **Examine repositories** inspect each system's repo configuration and verify a package's provenance.
- **Remove cleanly** remove software and clear orphaned dependencies.

# Key Takeaways

---

- **Lineage decides tooling** Rocky uses dnf/rpm; Ubuntu uses apt/dpkg — same concepts, different commands.
- **Live in the high-level tools** dnf and apt resolve dependencies automatically; reach for rpm/dpkg only for single files.
- **apt update is not dnf update** apt update refreshes the list; dnf update upgrades software.
- **Patching is security** keeping software current is your highest-impact routine control.
- **Trust the source** install from official, signed repositories and keep GPG checking on.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

# Questions?

---

Unit 6 · Package Management

